OVP Guide to Using Processor Models

Model specific information for RISC-V_RVI20U32mandatory

X-2025.12 December,2025

Synopsys®

Document Status

Author	Synopsys Inc
Version	X-2025.12
Filename	OVP_Model_Specific_Information_riscv_RVI20U32mandatory.pdf
Created	10 December 2025
Status	OVP Standard Release

Model Release Status

This model is included in all general releases.

Contents

Chapter 1

Overview

This document provides the details of an OVP Fast Processor Model variant.

OVP Fast Processor Models are written in C and provide a C API for use in C based platforms. The models also provide a native interface for use in SystemC TLM2 platforms.

The models are written using the OVP VMI API that provides a Virtual Machine Interface that defines the behavior of the processor. The VMI API makes a clear line between model and simulator allowing very good optimization and world class high speed performance. Most models are provided as a binary shared object and also as source. This allows the download and use of the model binary or the use of the source to explore and modify the model.

The models are run through an extensive QA and regression testing process and most model families are validated using technology provided by the processor IP owners. There is a companion document (OVP Guide to Using Processor Models) which explains the general concepts of OVP Fast Processor Models and their use. It is downloadable from the OVPworld website documentation pages.

1.1 Description

RISC-V RVI20U32mandatory 32-bit processor model

1.2 Licensing

This Model is released under the Open Source Apache 2.0

1.3 Extensions

1.3.1 Extensions Enabled by Default

The model has the following architectural extensions enabled, and the corresponding bits in the misa CSR Extensions field will be set upon reset:

misa bit 8: RV32I/RV64I/RV128I base integer instruction set

misa bit 20: extension U (User mode)

To specify features that can be dynamically enabled or disabled by writes to the misa register in addition to those listed above, use parameter "add_Extensions_mask". This is a string parameter containing the feature letters to add; for example, value "DV" indicates that double-precision floating point and the Vector Extension can be enabled or disabled by writes to the misa register, if supported on this variant. Parameter "sub_Extensions_mask" can be used to disable dynamic update of features in the same way.

Legacy parameter "misa_Extensions_mask" can also be used. This Uns32-valued parameter specifies all writable bits in the misa Extensions field, replacing any permitted bits defined in the base variant.

Note that any features that are indicated as present in the misa mask but absent in the misa will be ignored. See the next section.

1.3.2 Enabling Other Extensions

The following misa-visible extensions are supported by the model, but not enabled by default in this variant:

misa bit 0: extension A (atomic instructions)

misa bit 2: extension C (compressed instructions)

misa bit 3: extension D (double-precision floating point)

misa bit 4: RV32E base integer instruction set (embedded)

misa bit 5: extension F (single-precision floating point)

misa bit 7: extension H (hypervisor)

misa bit 12: extension M (integer multiply/divide instructions)

misa bit 13: extension N (user-level interrupts)

misa bit 15: extension P (DSP instructions)

misa bit 18: extension S (Supervisor mode)

misa bit 21: extension V (vector extension)

misa bit 23: extension X (non-standard extensions present)

To add features from this list to the visible set in the misa register, use parameter "add_Extensions". This is a string containing identification letters of features to enable; for example, value "DV" indicates that double-precision floating point and the Vector Extension should be enabled, if they are currently absent and are available on this variant.

Legacy parameter “misa_Extensions” can also be used. This Uns32-valued parameter specifies the reset value for the misa CSR Extensions field, replacing any permitted bits defined in the base variant.

The following implicit extensions are supported by the model, but not enabled by default in this variant:

implicit feature bit 1: extension B (bit manipulation extension)

implicit feature bit 10: extension K (cryptographic)

To add features from this list to the implicitly-enabled set (not visible in the misa register), use parameter “add_implicit_Extensions”. This is a string parameter in the same format as the “add_Extensions” parameter described above.

1.3.3 Disabling Extensions

The following misa-visible extensions are enabled by default in the model and can be disabled:

misa bit 20: extension U (User mode)

To disable features that are enabled by default, use parameter “sub_Extensions”. This is a string containing identification letters of features to disable; for example, value “DF” indicates that double-precision and single-precision floating point extensions should be disabled, if they are enabled by default on this variant.

1.4 General Features

1.4.1 mtvec CSR

On this variant, the Machine trap-vector base-address register (mtvec) is writable. It can instead be configured as read-only using parameter “mtvec_is_ro”.

Values written to “mtvec” are masked using the value 0xfffffffd. A different mask of writable bits may be specified using parameter “mtvec_mask” if required. In addition, on a write that enables vectored interrupt mode, parameter “tvec_align” may be used to specify additional hardware-enforced base address alignment on writes. In this variant, “tvec_align” defaults to 0, implying no additional alignment constraint on writes.

If parameter “mtvec_sext” is True, values written to “mtvec” are sign-extended from the most-significant writable bit. In this variant, “mtvec_sext” is False, indicating that “mtvec” is not sign-extended.

The initial value of “mtvec” is 0x0. A different value may be specified using parameter “mtvec” if required.

1.4.2 Reset

On reset, the model will restart at address 0x0. A different reset address may be specified using parameter "reset_address" or applied using optional input port "reset_addr" if required.

1.4.3 NMI

An NMI input is implemented. To instead specify that NMI is not implemented, set parameter "nmi_absent" to True.

On an NMI, the model will restart at address 0x0; a different NMI address may be specified using parameter "nmi_address" or applied using optional input port "nmi_addr" if required. The cause reported on an NMI is 0x0 by default; a different cause may be specified using parameter "ecode_nmi" or applied using optional input port "nmi_cause" if required.

If parameter "rnmi_version" is not "none", resumable NMIs are supported, managed by additional CSRs "mnscratch", "mnepc", "mncause" and "mnstatus", following the indicated version of the Resumable NMI extension proposal. In this variant, "rnmi_version" is "none".

On taking an NMI exception, the mstatus CSR will not be updated. To instead specify that mstatus fields mie and mpie should be updated upon taking an NMI exception, set parameter "nmi_update_mstatus" to True.

On taking an NMI exception, the tcontrol CSR will not be updated. To instead specify that tcontrol fields mte and mpte should be updated upon taking an NMI exception, set parameter "nmi_update_tcontrol" to True.

On taking an NMI exception, the mtval CSR will not be updated. To instead specify that mtval should be written with zero upon taking an NMI exception, set parameter "nmi_zero_mtval" to True.

The NMI input is level-sensitive. To instead specify that the NMI input is latched on the rising edge of the NMI signal, set parameter "nmi_is_latched" to True.

NMI interrupts are lower priority than Debug and Trigger Module events. To instead specify that NMI interrupts are higher priority, set parameter "nmi_high_priority" to True.

1.4.4 WFI

WFI will halt the processor until an interrupt occurs. It can instead be configured as a NOP by setting parameter "wfi_is_nop" to True.

The nominal time limit for WFI instructions can be set by parameter "TW_time_limit". In this variant, the time limit is 0 cycles.

Output signal "core_wfi_mode" indicates whether the processor is currently in WFI state.

Input signal "restart_wfi" will cause the processor to restart from WFI state when high.

Parameter "wfi_resume_not_trap" is 0 on this variant, meaning that pending wakeup events when WFI is executed will not prevent a trap occurring. if "wfi_resume_not_trap" is set to 1 then pending wakeup events when WFI is executed will cause the WFI to be treated as a NOP.

1.4.5 cycle CSR

The “cycle” CSR is not implemented in this variant and accesses will cause Illegal Instruction traps. Set parameter “cycle_undefined” to False to instead specify that “cycle” is implemented.

1.4.6 instret CSR

The “instret” CSR is not implemented in this variant and accesses will cause Illegal Instruction traps. Set parameter “instret_undefined” to False to instead specify that “instret” is implemented.

1.4.7 hpmcounter CSR

The “hpmcounter” CSRs are not implemented in this variant and accesses will cause Illegal Instruction traps. Set parameter “hpmcounter_undefined” to False to instead specify that “hpmcounter” CSRs are implemented.

1.4.8 time CSR

The “time” CSR is not implemented in this variant and reads of it will cause Illegal Instruction traps. Set parameter “time_undefined” to False to instead specify that “time” is implemented.

1.4.9 mcycle CSR

The “mcycle” CSR is implemented in this variant. Set parameter “mcycle_undefined” to True to instead specify that “mcycle” is unimplemented and accesses should cause Illegal Instruction traps.

1.4.10 minstret CSR

The “minstret” CSR is implemented in this variant. Set parameter “minstret_undefined” to True to instead specify that “minstret” is unimplemented and accesses should cause Illegal Instruction traps.

1.4.11 mhpmcounter CSR

The “mhpmcounter” CSRs are implemented in this variant. Set parameter “mhpmcounter_undefined” to True to instead specify that “mhpmcounter” CSRs are unimplemented and accesses should cause Illegal Instruction traps.

1.4.12 Unaligned Accesses

Unaligned memory accesses are not supported by this variant. Set parameter “unaligned” to “T” to enable such accesses.

Push/pop instructions will trigger address misaligned exceptions. Set parameter "unalignedPushPop" to "T" to enable misaligned accesses just for push/pop instructions.

Address misaligned exceptions are higher priority than page fault or access fault exceptions on this variant. Set parameter "unaligned_low_pri" to "T" to specify that they are lower priority instead.

1.4.13 PMP

A PMP unit is not implemented by this variant. Set parameter "PMP_registers" to indicate that the unit should be implemented with that number of PMP entries.

The total number of PMP registers present is 16, (this includes CSRs for unimplemented PMP regions), as determined by the requirements of the selected priv_version. Set parameter "PMP_csrs" to specify a different total number of PMP registers.

Accesses to unimplemented PMP registers are write-ignored and read as zero on this variant. Set parameter "PMP_undefined" to True to indicate that such accesses should cause Illegal Instruction exceptions instead.

1.4.14 Time and Timers

A RISC-V hart requires an incrementing time source to be available in any of the following cases:

1. The "time" CSR is implemented ("time_undefined" is False);
2. The "Sstc" extension is present ("Sstc" is True);
3. The internal CLINT model is enabled ("CLINT_address" is non-zero).

The time value in this model is implemented by an abstract counter object, which is shared with any other processor models or peripherals that use the same name to define the counter. The name of the counter is specified by the "mtime_counter" parameter ("mtime_counter" by default). The bit width of the counter is specified by the "mtime_bits" parameter (64 by default). The frequency of the counter is specified by the "mtime_Hz" parameter (1e+06Hz by default).

1.5 Privileged Architecture

This variant implements the Privileged Architecture with version specified in the References section of this document. Note that parameter "priv_version" can be used to select the required architecture version; see the following sections for detailed information about differences between each supported version.

1.5.1 Legacy Version 1.10

1.10 version of May 7 2017.

1.5.2 Version 20190608

Stable 1.11 version of June 8 2019, with these changes compared to version 1.10:

- mcountinhibit CSR defined;

- pages are never executable in Supervisor mode if page table entry U bit is 1;

- mstatus.TW is writable if any lower-level privilege mode is implemented (previously, it was just if Supervisor mode was implemented);

1.5.3 Version 20211203

1.12 draft version of December 3 2021, with these changes compared to version 20190608:

- mstatush, mseccfg, mseccfgh, menvcfg, menvcfgh, senvcfg, henvcfg, henvcfgh and mconfigptr CSRs defined;

- xret instructions clear mstatus.MPRV when leaving Machine mode if new mode is less privileged than M-mode;

- maximum number of PMP registers increased to 64;

- data endian is now configurable.

1.5.4 Version 1.12

Official 1.12 version, identical to 20211203.

1.5.5 Version 1.13

1.13 ratified version of October 17, 2024, with these changes compared to version 20190608:

- misa.MXL is now defined as read-only. The MXL_writable parameter no longer exists when priv_version >= 1.13;

- Defined the RV32-only medelegh and hedelegh CSRs;

- Defined hardware-error (19) and software-check (18) exception codes;

1.5.6 Version master

Unstable master version, currently identical to 1.13.

1.6 Unprivileged Architecture

This variant implements the Unprivileged Architecture with version specified in the References section of this document. Note that parameter "user_version" can be used to select the required architecture version; see the following sections for detailed information about differences between each supported version.

1.6.1 Legacy Version 2.2

2.2 version of May 7 2017.

1.6.2 Version 20191213

Stable 20191213-Base-Ratified version of December 13 2019, with these changes compared to version 2.2:

- floating point fmin/fmax instruction behavior modified to comply with IEEE 754-201x.

- numerous other optional behaviors can be separately enabled using Z-prefixed parameters.

1.7 Other Extensions

Other extensions that can be configured are described in this section.

1.7.1 Zicsr

Parameter "Zicsr" is 0 on this variant, meaning that standard CSRs and CSR access instructions are not implemented and an alternative scheme must be provided as a processor extension. if "Zicsr" is set to 1 then standard CSRs and CSR access instructions are implemented.

1.7.2 Zifencei

Parameter "Zifencei" is 0 on this variant, meaning that the fence.i instruction is not implemented. if "Zifencei" is set to 1 then the fence.i instruction is implemented (but treated as a NOP by the model).

1.7.3 Zihintpause

Parameter "Zihintpause" is 0 on this variant, meaning that the pause instruction is not implemented - instead the opcode will be interpreted as a FENCE instruction. if "Zihintpause" is set to 1 then the pause instruction is implemented. When executed and the PAUSE_time_limit parameter is not 0 it will pause the hart in WFI mode for up to the number of cycles specified by the time limit. When the time limit is 0 PAUSE is a NOP.

1.7.4 Zicbom

Parameter “Zicbom” is 0 on this variant, meaning that code block management instructions are undefined. if “Zicbom” is set to 1 then code block management instructions cbo.clean, cbo.flush and cbo.inval are defined.

If Zicbom is present, the cache block size is given by parameter “cmomp_bytes”. The instructions may cause traps if used illegally but otherwise are NOPs in this model.

1.7.5 Zicbop

Parameter “Zicbop” is 0 on this variant, meaning that prefetch instructions are undefined. if “Zicbop” is set to 1 then prefetch instructions prefetch.i, prefetch.r and prefetch.w are defined (but behave as NOPs in this model).

1.7.6 Zicboz

Parameter “Zicboz” is 0 on this variant, meaning that the cbo.zero instruction is undefined. if “Zicboz” is set to 1 then the cbo.zero instruction is defined.

If Zicboz is present, the cache block size is given by parameter “cmoz_bytes”.

1.7.7 Smdbltrp

Parameter “Smdbltrp” is 0 on this variant, meaning that M-mode double trap extensions are not implemented. if “Smdbltrp” is set to 1 then M-mode double trap extensions are implemented.

1.7.8 Smstateen

Parameter “Smstateen” is 0 on this variant, meaning that machine/supervisor/hypervisor state enable CSRs are undefined. if “Smstateen” is set to 1 then machine/supervisor/hypervisor state enable CSRs are defined.

Set Ssstateen to True and Smstateen to False to implement only the supervisor/hypervisor state enable CSRs.

1.7.9 Sscofpmf

Parameter “Sscofpmf” is 0 on this variant, meaning that count overflow and mode-based filtering extension CSRs are undefined. if “Sscofpmf” is set to 1 then count overflow and mode-based filtering extension CSRs are defined.

Note that this model implements CSR state only for this extension; hardware performance counters themselves are implementation-specific and not implemented.

1.7.10 Smcntrpmf

Parameter "Smcntrpmf" is 0 on this variant, meaning that cycle and instret mode-based filtering extension CSRs are undefined. if "Smcntrpmf" is set to 1 then cycle and instret mode-based filtering extension CSRs are defined.

1.7.11 Smcsrind

Parameter "Smcsrind" is 0 on this variant, meaning that indirect CSR access is not implemented. if "Smcsrind" is set to 1 then indirect CSR access is implemented. If Supervisor mode is implemented, this also implicitly enables Sscsrind.

1.7.12 Zawrs

Parameter "Zawrs" is 0 on this variant, meaning that wait-for-reservation-set instructions are not implemented. if "Zawrs" is set to 1 then wait-for-reservation-set instructions are implemented, in which case parameter "TW_time_limit" is used to specify the nominal cycle delay for wrs.nto, and parameter "STO_time_limit" is used to specify the nominal cycle delay for wrs.sto.

1.7.13 Zimop

Parameter "Zimop" is 0 on this variant, meaning that maybe operations are not implemented. if "Zimop" is set to 1 then maybe operations are implemented.

1.7.14 Zilsd/Zclsd Extensions for Load/Store pair on RV32

Parameter "Zilsd" is 0 on this variant, meaning that 32-bit load/store pair instructions for RV32 are not implemented. if "Zilsd" is set to 1 then 32-bit load/store pair instructions for RV32 are implemented.

1.7.15 Zibi

Parameter "Zibi" is 0 on this variant, meaning that branch-with-immediate instructions are not implemented. if "Zibi" is set to 1 then branch-with-immediate instructions are implemented.

1.8 CLIC

The model can be configured to implement a Core Local Interrupt Controller (CLIC) using parameter "CLICLEVELS"; when non-zero, the CLIC is present with the specified number of interrupt levels (2-256), as described in the RISC-V Core-Local Interrupt Controller specification, and further parameters are made available to configure other aspects of the CLIC. "CLICLEVELS" is zero in this variant, indicating that a CLIC is not implemented.

The model can be configured either to use an internal CLIC model (if parameter “externalCLIC” is False) or to present a net interface to allow the CLIC to be implemented externally in a platform component (if parameter “externalCLIC” is True). When the CLIC is implemented internally, net ports for standard interrupts and additional local interrupts are available. When the CLIC is implemented externally, a net port interface allowing the highest-priority pending interrupt to be delivered is instead present. This is described below.

1.8.1 CLIC Common Parameters

This section describes parameters applicable whether the CLIC is implemented internally or externally. These are:

“CLIC_version”: this defines the version of the CLIC specification that is implemented. See the References section of this document for more information.

“CLICANDBASIC”: this enum parameter indicates whether only the CLICinterrupt controller is present (if “NoBasic”) or both the CLIC and basic (AKA “CLINT”) interrupt controllers are present, with common interrupts shared between the two (if “BasicCommon”) or separate interrupts (if “BasicSeparate”). The default value in this variant is NoBasic.

“local_int_num”: This Uns32 parameter specifies the number of local interrupts implemented in the basic interrupt mode. These interrupts are numbered starting at 16, as the first 16 interrupts in the basic interrupt controller have architecturally defined meanings. The basic local interrupts correspond to bits 16 and above in the mie/mip CSRS. The maximum value for this parameter is 16 for RV32 and 48 for RV64. The default value in this variant is 0.

“NUM_INTERRUPT”: This Uns32 specifies the number of interrupts in the CLIC. If the clicinfo CSR is present then the num_interrupt field will be set to this value. The maximum value for this parameter is 4096. The minimum value for this parameter is 2, although setting it to 0 will set it to the local_int_num value, if that is greater than 2, for backwards compatibility. The default value in this variant is 0. Note that the clic_imp_list can also change the effective value used, by listing interrupt numbers above the specified value.

“CLICXNXTI”: this Boolean parameter indicates whether xnxti CSRs are implemented (if True) or unimplemented (if False). The default value in this variant is False.

“CLICXCSW”: this Boolean parameter indicates whether xscratchcsw and xscratchcswl CSRs registers are implemented (if True) or unimplemented (if False). The default value in this variant is False.

“tvt_undefined”: this Boolean parameter indicates whether xtvt CSRs registers are implemented (if True) or unimplemented (if False). If the registers are unimplemented then the model will use basic mode vectored interrupt semantics based on the xtvec CSRs instead of Selective Hardware Vectoring semantics described in the specification. The default value in this variant is False.

“intthresh_undefined”: this Boolean parameter indicates whether xintthresh CSRs are implemented (if True) or unimplemented (if False). The default value in this variant is False.

“INTTHRESHBITS”: if xintthresh CSRs are implemented, this parameter indicates the number of implemented bits in the “th” field. The default value in this variant is 0.

"mclicbase_undefined": this Boolean parameter indicates whether the mclicbase CSR register is implemented (if True) or unimplemented (if False); for CLIC versions after 0.9-draft-20220315, this feature is not configurable and this parameter is always True. The default value in this variant is True.

1.8.2 CLIC Internal-Implementation Parameters

This section describes parameters applicable only when the CLIC is implemented internally. These are:

"mclicbase": this parameter specifies the CLIC base address for M-mode interrupts in physical memory. The default value in this variant is 0x0.

"CLICCFGMBITS": this Uns32 parameter indicates the maximum number of bits that can be specified as implemented in cliccfg.nmbits, and also indirectly defines CLICPRIVMODES. For cores which implement only Machine mode, or which implement only Machine and User modes but not the N extension, the parameter is absent ("CLICCFGMBITS" must be zero in these cases). The default value in this variant is 0.

"CLICCFGLBITS": this Uns32 parameter indicates the maximum number of bits that can be specified as implemented in cliccfg.nlbits. The default value in this variant is 0. See also parameter "nlbits_valid" which allows fine-grain specification of legal values in cliccfg.nlbits.

"nlbits_valid": this Uns32 parameter is a bitmask of legal values that can be specified as implemented in the WARL cliccfg.nlbits field. As an example, a value of 8 in cliccfg.nlbits is legal only if bit 8 is set in nlbits_valid. The default value of "nlbits_valid" in this variant is 0x0.

"CLICSELHVEC": this Boolean parameter indicates whether Selective Hardware Vectoring is supported (if True) or unsupported (if False). The default value in this variant is False.

"forceCLICmmio": this Boolean parameter indicates whether the MMIO interface is present when CLIC_version is set to version 0.10-draft-20250317 or later (it is ignored for earlier versions.) As of CLIC version 0.10 the indirect CSR method is required, but the MMIO may optionally be implemented also, by setting this parameter to True. The default value in this variant is False.

"force_clicinfo": this Boolean parameter forces the clicinfo register to be present. This has no effect when the CLIC_version selection is earlier than 0.9.20221108, where clicinfo is always prsent. The default value in this variant is False.

1.8.3 CLIC Individual Interrupt Configuration Parameters

The following behaviors are individually configurable for CLIC interrupts using the set of parameters described in this section:

- whether the interrupt is implemented or not. Registers for unimplemented interrupts are read-only zeros, writes ignored, and no input port is created for them.

- whether the clicintattr.MODE field is read-write or read-only, and whether it is initialized to Machine (3) or Supervisor (1).

- whether the clicintattr.TRIG field is read-write or read-only, and whether it is initialized to

POSLEVEL (0), POSEDGE (1), NEGLEVEL (2) or NEGEDGE (3).

The following two "clic_X_default" parameters are used to define the default behavior of the clicintattr MODE and TRIG fields. This will be the behavior for any interrupt not included in any of the "clic_X_list" parameters described later.

"clic_mode_default": this enum parameter specifies the default behavior for the clicintattr.MODE field for all implemented CLIC interrupts. The supported values are "RW_Machine", "RW_Supervisor", "RO_Machine" or "RO_Supervisor". The default value in this variant is RW_Machine.

"clic_trig_default": this enum parameter specifies the default behavior for the clicintattr.TRIG field for all implemented CLIC interrupts. The supported values are "RW_" or "RO_", followed by "POSLEVEL", "POSEDGE", "NEGLEVEL" or "NEGEDGE". The default value in this variant is RW_POSLEVEL.

The following parameters are all comma-separated lists of interrupt numbers (they may be decimal numbers or hex numbers preceded by "0x"). White space is permitted. The largest number allowed is 4095, the highest interrupt number supported by the CLIC architecture. It is a fatal error to specify an invalid list.

The behavior is undefined when the same interrupt number is specified in multiple lists that configure conflicting behavior.

"clic_imp_list" and "clic_unimp_list": interrupts 0..NUM_INTERRUPTS-1 are implemented by default, and these list parameters are used to refine that default configuration further. Interrupt numbers listed in "clic_imp_list" will be implemented, while those in "clic_unimp_list" will not be implemented. The default values in this variant are clic_imp_list="" and clic_unimp_list="".

"clic_mode_RO_list", "clic_mode_RW_list", "clic_mode_M_list" and "clic_mode_S_list": These are used to override the default behavior of the clicintattr.MODE field, both whether the field is writable and the initial value for the field. The default values in this variant are clic_mode_RO_list="", clic_mode_RW_list="", clic_mode_M_list="", and clic_mode_S_list="".

"clic_trig_RO_list", "clic_trig_RW_list", "clic_trig_posedge_list" and "clic_trig_negedge_list", "clic_trig_poslevel_list" and "clic_trig_neglevel_list": These are used to override the default behavior of the clicintattr.TRIG field, both whether the field is writable and the initial value for the field. The default values in this variant are clic_trig_RO_list="", clic_trig_RW_list="", clic_trig_posedge_list="", clic_trig_negedge_list="", clic_trig_poslevel_list="", and clic_trig_neglevel_list="".

1.8.4 CLIC Deprecated Parameters

The following parameters are deprecated in favor of the more general-purpose "clic_X_list" parameters described above. Everything that is configurable by the deprecated parameters can and should instead be done using those updated parameters.

"posedge_0_63" (deprecated): this Uns64 parameter specifies a mask for CLIC interrupts 0 to 63 indicating whether an interrupt is fixed positive-edge triggered. If the corresponding mask bit for interrupt N is 1 then the clicintattr.trig field for that interrupt is read-only and initialized appropriately. If zero, then the trig field is either fixed level triggered (see below) or is writable.

The default value in this variant is posedge_0_63=0x0.

"posedge_other" (deprecated): this Boolean parameter indicates whether CLIC interrupts 64 and above are fixed positive-edge triggered. If True then the clicintattr.trig fields for these interrupts are read-only and initialized appropriately. If False, then the trig fields are either fixed level triggered (see below) or are writable. The default value in this variant is posedge_other=False.

"poslevel_0_63" (deprecated): this Uns64 parameter specifies a mask for CLIC interrupts 0 to 63 indicating whether an interrupt is fixed positive-level triggered. If the corresponding mask bit for interrupt N is 1 then the clicintattr.trig field for that interrupt is read-only and initialized appropriately. If zero, then the trig field is writable. This is ignored when the interrupt is configured as fixed positive-edge triggered (see above). The default value in this variant is poslevel_0_63=0x0.

"poslevel_other" (deprecated): this Boolean parameter indicates whether CLIC interrupts 64 and above are fixed positive level triggered. If True then the clicintattr.trig fields for these interrupts are read-only and initialized appropriately. If False, then the trig fields are writable. This is ignored when the other interrupts are configured as fixed positive-edge triggered (see above). The default value in this variant is poslevel_other=False.

"CSIP_present" (deprecated): this Boolean parameter indicates whether the CSIP interrupt is implemented (if True) or unimplemented (if False). If implemented, the interrupt is positive-edge triggered, and has id 12 or 16, depending on the CLIC version selected. The default value in this variant is False.

1.8.5 CLIC External-Implementation Net Port Interface

When the CLIC is externally implemented, net ports are present allowing the external CLIC model to supply the highest-priority pending interrupt and to be notified when interrupts are handled. These are:

"irq_id_i": this input should be written with the id of the highest-priority pending interrupt.

"irq_lev_i": this input should be written with the highest-priority interrupt level.

"irq_sec_i": this 2-bit input should be written with the highest-priority interrupt security state (00:User, 01:Supervisor, 11:Machine).

"irq_shv_i": this input port should be written to indicate whether the highest-priority interrupt should be direct (0) or vectored (1). If the "tvt_undefined parameter" is False, vectored interrupts will use selective hardware vectoring, as described in the CLIC specification. If "tvt_undefined" is True, vectored interrupts will behave like basic mode vectored interrupts.

"irq_i": this input should be written with 1 to indicate that the external CLIC is presenting an interrupt, or 0 if no interrupt is being presented.

"irq_ack_o": this output is written by the model on entry to the interrupt handler (i.e. when the interrupt is taken). It will be written as an instantaneous pulse (i.e. written to 1, then immediately 0).

"irq_id_o": this output is written by the model with the id of the interrupt currently being handled. It is valid during the instantaneous irq_ack_o pulse.

“sec_lvl_o”: this output signal indicates the current secure status of the processor, as a 2-bit value (00=User, 01:Supervisor, 11=Machine).

1.8.6 CLIC Versions

The CLIC specification has been under active development. To enable simulation of hardware that may be based on an older version of the specification, the model implements behavior for a number of previous versions of the specification. The differing features of these are listed below, in chronological order.

1.8.7 Version 20180831

Legacy version of August 31 2018, compatible with early SiFive cores.

1.8.8 Version 0.9-draft-20191208

Stable 0.9 version of December 8 2019.

1.8.9 Version 0.9-draft-20220315

Stable 0.9 version of March 15 2022, with these changes compared to version 0.9-draft-20191208:

- level-sensitive interrupts may no longer be set pending by software;

- if there is an access exception on table load when Selective Hardware Vectoring is enabled, both xtval and xepc hold the faulting address;

- if the xinhv bit is set, the target address for an xret instruction is obtained by a load from address xepc.

1.8.10 Version 0.9-draft-20221108

Stable 0.9 version of November 8 2022, with these changes compared to version 0.9-draft-20191208:

- vector table reads now require execute permission (not read permission);

- CSIP now has id 16 and is treated as the first local interrupt (previously, it had id 12 and was not considered to be a local interrupt);

- algorithm used for xscratchcsw accesses changed;

- new parameter INTTHRESHBITS allows implemented bits in xintthresh CSRs to be restricted.

- whether interrupt handling is in CLIC or CLINT mode is now determined by mtvec.MODE at all privilege levels (when both modes are supported).

- xintstatus registers have been relocated at read-only CSR addresses (e.g. 0xF46 for mintstatus).

- cliccfg.nvbits field has been removed.

- clicinfo memory-mapped register has been removed.

1.8.11 Version 0.9-draft-20230801

Stable 0.9 version of August 1 2023, with these changes compared to version 0.9-draft-20221108:

- xintstatus registers have been relocated at different read-only CSR addresses (e.g. 0xFB1 for mintstatus);

- 32-bit xcliccfg registers are present on a configuration page preceding interrupt status pages, containing separate unlbits, snlbits and mnlbits fields for each mode;

- if the hart is currently running at privilege mode x, an mret, sret, dret or mnret instruction that changes privilege mode sets xintthresh=0;

- vector table reads now require execute permission (not read permission);

- xepc is forced to table-entry alignment on xRET when inhv is active;

- inhv is now held in exception mode, not interrupt mode.

1.8.12 Version 0.10-draft-20250317

Draft version of March 17 2025, with these changes compared to version 0.9-draft-20230801:

- Indirect CSRs replace the CLIC Memory-Mapped Registers. (new parameter "forceCLICmmio" added to implement MMIO registers in addition to indirect CSRs.)

- CSRs and parameters associated with suclic extension are no longer present. The proposed User mode interrupt N extension is no longer allowed to be enabled when a CLIC version 0.10 or later is present.

- Add mandatory reset to 0 of mcause.mpil.

- CLIC is allocated bit 53 in the stateen0 register to block access to the ssclic CSRs.

1.8.13 Version master

Unstable master version as of 17 March 2025 (commit 62092fc)

1.9 Advanced Interrupt Architecture

The model can be configured to implement the Advanced Interrupt Architecture (AIA) interface using Boolean parameter "Smaia"; when True, the AIA interface is present as described in the RISC-V Advanced Interrupt Architecture specification, and further parameters are made available to configure other aspects of the interface. "Smaia" is False in this variant, indicating that the AIA interface is not implemented.

1.10 WorldGuard Extension

The model can be configured to implement the WorldGuard ISA extension using parameter "WG_version"; when not "none", the WorldGuard ISA extension is present as described in the RISC-V WorldGuard specification, and further parameters are made available to configure other aspects of the extension. "WG_version" is "none" in this variant, indicating that WorldGuard ISA extension is not implemented.

1.11 Interrupts

The "reset" port is an active-high reset input. The processor is halted when "reset" goes high and resumes execution from the reset address specified using the "reset_address" parameter or "reset_addr" port when the signal goes low. The "mcause" register is cleared to zero.

The "nmi" port is an active-high NMI input. The processor resumes execution from the address specified using the "nmi_address" parameter or "nmi_addr" port when the NMI signal goes high. The "mcause" register is cleared to zero.

All other interrupt ports are active high. For each implemented privileged execution level, there are by default input ports for software interrupt, timer interrupt and external interrupt; for example, for Machine mode, these are called "MSWInterrupt", "MTimerInterrupt" and "MExternalInterrupt", respectively. When the N extension is implemented, ports are also present for User mode. Parameter "unimp_int_mask" allows the default behavior to be changed to exclude certain interrupt ports. The parameter value is a mask in the same format as the "mip" CSR; any interrupt corresponding to a non-zero bit in this mask will be removed from the processor and read as zero in "mip", "mie" and "mideleg" CSRs (and Supervisor and User mode equivalents if implemented).

Parameter "external_int_id" can be used to enable extra interrupt ID input ports on each hart. If the parameter is True then when an external interrupt is taken the value on the ID port is sampled and used to fill the Exception Code field in the relevant "xcause" CSR. For Machine External interrupts, the extra interrupt ID port is called "MExternalInterruptID"; for Supervisor External interrupts, the extra interrupt ID port is called "SExternalInterruptID".

The "deferint" port is an active-high artifact input that, when written to 1, prevents any pending-and-enabled interrupt being taken (normally, such an interrupt would be taken on the next instruction after it becomes pending-and-enabled). The purpose of this signal is to enable alignment with hardware models in step-and-compare usage.

1.12 Triggering Exceptions Externally

The following artifact input net port(s) are provided to support triggering exceptions externally (e.g. by a DV test harness or an external bus controller model.)

illegalinstruction - This port is used to immediately trigger an Illegal instruction exception (exception code 2). The exception is triggered when a leading edge (0->1 transition) is seen on the net. If the processor is in a WFI state with a bounded time limit enabled, then the illegal instruction exception follows the conventions for an event causing the hart to resume from WFI mode, namely the EPC will be set to the instruction following the WFI instruction when taking the exception.

1.13 Debug Mode (Sdext)

The model can be configured to implement Debug mode using parameter "debug_mode". This implements features described in Chapter 4 of the RISC-V External Debug Support specification with version specified by parameter "debug_version" (see References). Some aspects of this mode are not defined in the specification because they are implementation-specific; the model provides infrastructure to allow implementation of a Debug Module using a custom harness. Features added are described below.

Parameter "debug_mode" can be used to specify four different behaviors, as follows:

1. If set to value "vector", then operations that would cause entry to Debug mode result in the processor jumping to the address specified by the "debug_address" parameter. It will execute at this address, in Debug mode, until a "dret" instruction causes return to non-Debug mode. Any exception generated during this execution will cause a jump to the address specified by the "dexc_address" parameter.

2. If set to value "interrupt", then operations that would cause entry to Debug mode result in the processor simulation call (e.g. opProcessorSimulate) returning, with a stop reason of OP_SR_INTERRUPT. In this usage scenario, the Debug Module is implemented in the simulation harness.

3. If set to value "halt", then operations that would cause entry to Debug mode result in the processor halting. Depending on the simulation environment, this might cause a return from the simulation call with a stop reason of OP_SR_HALT, or debug mode might be implemented by another platform component which then restarts the debugged processor again.

4. If set to value "inject", then operations that would cause entry to Debug mode result in the processor continuing to execute from the current address in Debug mode. The harness should detect that Debug mode has been entered by monitoring the "DM" integration support register, and inject Debug-mode instructions one at a time using function opProcessorSimulateInstruction. Debug mode is exited by either an explicit write of False to the "DM" register or by execution of an injected "dret" instruction, as described by "Debug State Exit" below.

Note that parameter "debug_mode" can be also be set to "none" to specify that Sdext is not implemented.

1.13.1 Debug State Entry

The specification does not define how Debug mode is implemented. In this model, Debug mode is enabled by a Boolean pseudo-register, "DM". When "DM" is True, the processor is in Debug mode. When "DM" is False, mode is defined by "mstatus" in the usual way.

Entry to Debug mode can be performed in any of these ways:

1. By writing True to register "DM" (e.g. using opProcessorRegWrite) followed by simulation of at least one cycle (e.g. using opProcessorSimulate) - in this case, dcsr.cause will report a cause of

trigger (2);

2. By writing a 1 then 0 to net "haltreq" (using opNetWrite) followed by simulation of at least one cycle (e.g. using opProcessorSimulate) - in this case, dcsr.cause will report a cause of haltreq (3);

3. By writing a 1 to net "resethaltreq" (using opNetWrite) while the "reset" signal undergoes a negedge transition, followed by simulation of at least one cycle (e.g. using opProcessorSimulate) - in this case, dcsr.cause will report a cause of resethaltreq (5) or haltreq (3), depending on the value of parameter "no_resethaltreq";

4. By executing an "ebreak" instruction when Debug mode entry for the current processor mode is enabled by dcsr.ebreakm, dcsr.ebreaks or dcsr.ebreaku - in this case, dcsr.cause will report a cause of ebreak (1);

5. By executing a single instruction when Debug mode entry for the current processor mode is enabled by dcsr.step - in this case, dcsr.cause will report a cause of step (4);

6. By a Trigger Module trigger, when that trigger is configured to enter Debug mode - in this case, dcsr.cause will report a cause of trigger (2).

In all cases, the processor will save required state in "dpc" and "dcsr" and then perform actions described above, depending in the value of the "debug_mode" parameter.

1.13.2 Debug State Exit

Exit from Debug mode can be performed in either of these ways:

1. By writing False to register "DM" (e.g. using opProcessorRegWrite) followed by simulation of at least one cycle (e.g. using opProcessorSimulate);

2. By executing an "dret" instruction when Debug mode.

In both cases, the processor will perform the steps described in section 4.6 (Resume) of the Debug specification.

1.13.3 Debug Registers

When Debug mode is enabled, registers "dcsr" and "dpc" are implemented as described in the specification. Registers "dscratch0" and "dscratch1" may also be implemented (see parameters below). Implemented registers may be manipulated externally by a Debug Module using opProcessorRegRead or opProcessorRegWrite; for example, the Debug Module could write "dcsr" to enable "ebreak" instruction behavior as described above, or read and write "dpc" to emulate stepping over an "ebreak" instruction prior to resumption from Debug mode.

Parameter "dscratch0_undefined" is used to specify whether the "dscratch0" CSR is undefined, in which case accesses to it trap to Machine mode. In this variant, "dscratch0_undefined" is 0.

Parameter "dscratch1_undefined" is used to specify whether the "dscratch1" CSR is undefined, in which case accesses to it trap to Machine mode. In this variant, "dscratch1_undefined" is 0.

Parameter dcsr_stepie is used to specify the behavior of the dcsr.stepie field. This is currently configured as read/write, reset to 0.

Parameter dcsr_stopcount is used to specify the behavior of the dcsr.stopcount field. This is currently configured as read/write, reset to 0.

Parameter dcsr_stoptime is used to specify the behavior of the dcsr.stoptime field. This is currently configured as read/write, reset to 0.

Parameter dcsr_mprven is used to specify the behavior of the dcsr.mprven field. This is currently configured as read/write, reset to 0.

1.13.4 Debug Mode Execution

The specification allows execution of code fragments in Debug mode. A Debug Module implementation can cause execution in Debug mode by the following steps:

1. Write the address of a Program Buffer to the program counter using opProcessorPCSet;

2. If "debug_mode" is set to "halt", write 0 to pseudo-register "DMStall" (to leave halted state);

3. If entry to Debug mode was handled by exiting the simulation callback, call opProcessorSimulate or opRootModuleSimulate to resume simulation.

Debug mode will be re-entered in these cases:

1. By execution of an "ebreak" instruction; or:

2. By execution of an instruction that causes an exception.

In both cases, the processor will either jump to the debug exception address, or return control immediately to the harness, with stopReason of OP_SR_INTERRUPT, or perform a halt, depending on the value of the "debug_mode" parameter.

1.13.5 Debug Single Step

When in Debug mode, the processor or harness can cause a single instruction to be executed on return from that mode by setting dcsr.step. After one non-Debug-mode instruction has been executed, control will be returned to the harness. The processor will remain in single-step mode until dcsr.step is cleared.

1.13.6 Debug Event Priorities

The model supports three different models for determining which debug exception occurs when step, execute address, resethaltreq and haltreq events are all pending. These options are listed below, with highest-priority event first:

1. when parameter "debug_priority"="sxh": step ->execute address ->resethaltreq ->haltreq;

2. when parameter "debug_priority"="shx": step ->resethaltreq ->haltreq ->execute address;

3. when parameter "debug_priority"="hsx": resethaltreq ->haltreq ->step ->execute address.

1.13.7 Debug Ports

Port "DM" is an output signal that indicates whether the processor is in Debug mode

Port "haltreq" is a rising-edge-triggered signal that triggers entry to Debug mode (see above).

Port "resethaltreq" is a level-sensitive signal that triggers entry to Debug mode after reset (see above).

1.13.8 Debug Mode Versions

Debug mode specification has been under active development. To enable simulation of hardware that may be based on an older version of the specification, the model implements behavior for a number of versions of the specification. The differing features of these are listed below, in chronological order.

1.13.9 Version 0.13.2-DRAFT

0.13.2-DRAFT version of March 22 2019.

1.13.10 Version 0.14.0-DRAFT

0.14.0-DRAFT version of November 6 2020.

1.13.11 Version 1.0.0-STABLE

1.0.0-STABLE version of February 9 2022.

1.13.12 Version 1.0-STABLE

1.0-STABLE version of December 28 2022, with these changes compared to version 1.0.0-STABLE:

- nmi is moved from etrigger to itrigger and is now subject to the mode bits in that trigger.

1.14 Debug Mask

It is possible to enable model debug messages in various categories. This can be done statically using the "debugflags" parameter, or dynamically using the "debugflags" command. Enabled messages are specified using a bitmask value, as follows:

Value 0x002: enable debugging of PMP and virtual memory state;

Value 0x004: enable debugging of interrupt state;

All other bits in the debug bitmask are reserved and must not be set to non-zero values.

1.15 Integration Support

This model implements a number of non-architectural pseudo-registers, commands, and other features to facilitate integration.

1.15.1 Command "setPMA -attributes <attrs>-lo <addr>-hi <addr>"

This command allows PMA attributes to be set for the address range lo:hi. The required attributes are described by the "attrs" string, which can contain any combination of these characters:

"r": allow read access

"w": allow write access

"x": allow execute access

"a": disallow unaligned accesses

"A": disallow RVMC_USER1 accesses (often AMO and LR/SC)

"P": disallow RVMC_USER2 accesses (often push/pop)

"1": allow 1-byte accesses (if none of 1248 are specified then all are allowed)

"2": allow 2-byte accesses

"4": allow 4-byte accesses

"8": allow 8-byte accesses

<space>, "-": ignored, use for formatting

The command may be used multiple times, in which case PMA attributes for later commands override those specified for earlier ones where ranges overlap. A common idiom is to deny all access to the entire memory range in the first command before adding back permissions for subregions with subsequent commands.

1.15.2 Command "getCSRIndex -name <name>"

This command returns the index number of a named CSR, or -1 if that CSR does not exist.

1.15.3 Command "listCSRs"

This command lists all implemented CSRs in index order.

1.15.4 CSR Register External Implementation

If parameter "enable_CSR_bus" is True, an artifact 16-bit bus "CSR" is enabled. Slave callbacks installed on this bus can be used to implement modified CSR behavior (use opBusSlaveNew or icmMapExternalMemory, depending on the client API). A CSR with index 0xABC is mapped on the bus at address 0xABC0; as a concrete example, implementing CSR "time" (number 0xC01) externally requires installation of a read callback at address 0xC010 on the CSR bus.

If both read and write callbacks are installed, or if a read callback is installed and the CSR is in the read-only address space, then the read callback will be used to provide the value for both true accesses and for trace and API register read (using opRegRead, etc). However, if only a read callback is installed and the CSR is in the CSR read/write address space then the callback will be used for true register reads *only*; in this case, the *model* CSR implementation will be used for trace and API register read. This idiom allows values to be injected for volatile CSRs without changing fundamental model behavior.

An artifact net, "readcsr", can also be used to override the value apparently read from a CSR without resorting to the CSR bus. When a CSR is read into a GPR that is not "x0", this net is written with a value encoding the CSR number (in bits 11:0) and destination GPR number (in bits 20:16). To use this net:

1. Install a net monitor callback on "readcsr" using opNetWriteMonitorAdd;

2. When the callback is activated, extract the encoded CSR and GPR numbers;

3. If the CSR number corresponds to a CSR of interest, find the OP register corresponding to the GPR using opProcessorRegByIndex;

4. Use opProcessorRegWrite to modify the GPR value.

1.15.5 External Stimulation of Illegal Instruction Trap

Artifact input net port "illegalinstr" allows Illegal Instruction traps to be raised externally. On a rising edge of the signal connected to this port, the hart will immediately take an Illegal Instruction trap with "xepc" set to the current program counter.

As a special case, if the hart is currently stalled by a WFI instruction ("wfi_is_nop" is False), it will be restarted and take either an Illegal Instruction or Virtual Instruction trap, based on the current processor mode and the governing TW bit.

1.15.6 Halt Reason Introspection

An artifact register HaltReason can be read to determine the reason or reasons that a processor is halted. This register is a bitfield, with the following encoding:

0x001: waiting in WFI

0x002: waiting in reset

0x004: waiting for debug

0x008: waiting for reservation set (wrs.*)

0x010: waiting for reservation set (wrs.sto)

0x020: waiting for any pending and enabled interrupt

0x040: waiting for custom reason with WFI semantics

0x080: waiting for custom reason with NMI semantics

0x100: waiting for custom reason (persists over reset)

0x200: waiting because of critical error (Smdbltrp)

An output net of the same name is also present. The net is written whenever the HaltReason changes.

1.16 Instruction Disassembly

This model implements a number of parameters to control instruction disassembly, as shown in trace output.

If parameter "use_hw_reg_names" is True, instruction disassembly shows hardware names x0-x31. If "use_hw_reg_names" is False, ABI names are shown instead.

If parameter "no_pseudo_inst" is True, instruction disassembly always shows true instructions. If "no_pseudo_inst" is False, pseudo-instructions are shown instead where applicable.

If parameter "show_c_prefix" is True, instruction disassembly of 16-bit instructions will include a compressed prefix (e.g. "c." or "cm."). If "show_c_prefix" is False, the compressed prefix will be omitted.

1.17 Semihosting

Extension libraries for semihosting of the host computer I/O and related facilities are available. A variety of protocols are supported as described below. These are all in the VLNV tree with vendor=riscv.ovpworld.org and library=semihosting.

By default, the pk semihosting (see below) is enabled. To select a different riscv.ovpworld.org semihost library, use the "semihostname" simulator parameter.

To disable semihosting entirely, for example when running an operating system where drivers are implemented along with I/O hardware emulated by platform peripherals, override the"defaultsemihost" processor parameter with the value True.

1.17.1 Proxy Kernel semihosting (default)

The Riscv Proxy Kernel semihosting Execution Environment Interface (EEI) is supported by the riscv.ovpworld.org semihosting library with name=pk. This uses the ecall or ebreak instruction with register a7 (on RVI) or x5 (on RVE) to hold the call type, while registers a0 thru a6 (on RVI) or x10 thru x15 (on RVE) hold argument values. See https://github.com/riscv-software-src/riscv-pk for details.

When using the Riscv GNU gcc toolchain this corresponds to using the command line option –specs=sim.specs (which is usually configured as the default behavior.)

1.17.2 Angel-compatible semihosting

An Angel-compatible semihosting Execution Environment Interface (EEI) is supported by the riscv.ovpworld.org semihosting library with name=semihost. This uses the ecall or ebreak instruction with register a0 to hold the call type, while register a1 contains a pointer to a data block with the argument values for that call type.

When using the Riscv GNU gcc toolchain this corresponds to using the command line option –specs=semihost.specs. See libgloss/riscv in https://sourceware.org/git/?p=newlib-cygwin.git;a=tree for details.

1.17.3 Newlib interception semihosting

Semihosting that intercepts newlib calls directly is supported by the riscv.ovpworld.org semihosting library with name=riscv32Newlib or riscv64Newlib. Using this semihost library removes any dependency on the EEI protocol implemented by the libgloss layer, but does not provide complete testing of the toolchain code or provide support for direct environment calls that do not go through newlib functions.

1.18 Limitations

Instruction pipelines are not modeled in any way. All instructions are assumed to complete immediately. This means that instruction barrier instructions (e.g. fence.i) are treated as NOPs, with the exception of any Illegal Instruction behavior, which is modeled.

Caches and write buffers are not modeled in any way. All loads, fetches and stores complete immediately and in order, and are fully synchronous. Data barrier instructions (e.g. fence) are treated as NOPs, with the exception of any Illegal Instruction behavior, which is modeled.

Real-world timing effects are not modeled: all instructions are assumed to complete in a single cycle.

Hardware Performance Monitor registers are not implemented and hardwired to zero.

The internal CLIC does not model the optional interrupt triggers (clicinttrig registers.) These registers appear as hard-wired zeros.

1.19 Verification

All instructions have been extensively tested by Imperas, using tests generated specifically for this model and also reference tests from https://github.com/riscv/riscv-tests.

Also reference tests have been used from various sources including:

https://github.com/riscv/riscv-tests

https://github.com/ucb-bar/riscv-torture

The Imperas OVPsim RISC-V models are used in the RISC-V Foundation Compliance Framework as a functional Golden Reference:

https://github.com/riscv/riscv-compliance

where the simulated model is used to provide the reference signatures for compliance testing. The Imperas OVPsim RISC-V models are used as reference in both open source and commercial instruction stream test generators for hardware design verification, for example:

http://valtrix.in/sting from Valtrix

https://github.com/google/riscv-dv from Google

The Imperas OVPsim RISC-V models are also used by commercial and open source RISC-V Core RTL developers as a reference to ensure correct functionality of their IP.

1.20 References

The Model details are based upon the following specifications:

RISC-V Instruction Set Manual, Volume I: User-Level ISA (User Architecture Version 20191213)

RISC-V Instruction Set Manual, Volume II: Privileged Architecture (Privileged Architecture Version 1.11 - ratified 20190608)

Implements only the mandatory extensions for RVI20U32 as listed in:

RISC-V Profiles Version 1.0, April 2, 2023

Chapter 2

Configuration

2.1 Location

This model's VLNV is riscv.ovpworld.org/processor/riscv/1.0.
The model source is usually at:
$IMPERAS_HOME/ImperasLib/source/riscv.ovpworld.org/processor/riscv/1.0
The model binary is usually at:
$IMPERAS_HOME/lib/$IMPERAS_ARCH/ImperasLib/riscv.ovpworld.org/processor/riscv/1.0

2.2 GDB Path

The default GDB for this model is: $IMPERAS_HOME/lib/$IMPERAS_ARCH/gdb/riscv-none-embed-gdb.

2.3 Semi-Host Library

The default semi-host library file is riscv.ovpworld.org/semihosting/pk/1.0

2.4 Processor Endian-ness

This is a LITTLE endian model.

2.5 QuantumLeap Support

This processor is qualified to run in a QuantumLeap enabled simulator.

2.6 Processor ELF code

The ELF code supported by this model is: 0xf3.

Chapter 3

All Variants in this model

This model has these variants

Variant	Description
RV32I	
RV32IM	
RV32IMC	
RV32IMCZce	
RV32IMAC	
RV32G	
RV32GC	
RV32GCZfinx	
RV32GCB	
RV32GCH	
RV32GCK	
RV32GCN	
RV32GCV	
RV32E	
RV32EC	
RV32EM	
RV64I	
RV64IM	
RV64IMC	
RV64IMCZce	
RV64IMAC	
RV64G	
RV64GC	
RV64GCZfinx	
RV64GCB	
RV64GCH	
RV64GCK	
RV64GCN	
RV64GCV	
RVB32I	
RVB32E	

RVB64I	
RVI20U32mandatory	(described in this document)
RVI20U32	
RVI20U64mandatory	
RVI20U64	
RVA20U64mandatory	
RVA20U64	
RVA20S64mandatory	
RVA20S64	
RVA22U64mandatory	
RVA22U64	
RVA22S64mandatory	
RVA22S64	
RVA23U64mandatory	
RVA23U64	
RVA23S64mandatory	
RVA23S64	
RVB23U64mandatory	
RVB23U64	
RVB23S64mandatory	
RVB23S64	

Table 3.1: All Variants in this model

Chapter 4

Bus Master Ports

This model has these bus master ports.

Name	min	max	Connect?	Description
INSTRUCTION	32	34	mandatory	Instruction bus
DATA	32	34	optional	Data bus

Table 4.1: Bus Master Ports

Chapter 5

Bus Slave Ports

This model has no bus slave ports.

Chapter 6

Net Ports

This model has these net ports.

Name	Type	Connect?	Description
reset	input	optional	Reset
reset_addr	input	optional	Externally-applied reset address
nmi	input	optional	NMI
nmi_cause	input	optional	Externally-applied NMI cause
nmi_addr	input	optional	Externally-applied NMI address
MSWInterrupt	input	optional	Machine software interrupt
MTimerInterrupt	input	optional	Machine timer interrupt
MExternalInterrupt	input	optional	Machine external interrupt
irq_ack_o	output	optional	Interrupt acknowledge (pulse)
irq_id_o	output	optional	Acknowledged interrupt id (valid during irq_ack_o pulse)
sec_lvl_o	output	optional	Current privilege level
illegalinstr	input	optional	Artifact signal causing Illegal Instruction exception on rising edge
deferint	input	optional	Artifact signal causing interrupts to be held off while high
coverpoint	output	optional	Artifact port written with coverage point identifier
readcsr	output	optional	Artifact port written with CSR/GPR information when CSR is read
HaltReason	output	optional	Artifact halt reason indication
core_wfi_mode	output	optional	WFI active indication
restart_wfi	input	optional	Artifact signal causing restart from WFI state when high

Table 6.1: Net Ports

Chapter 7

FIFO Ports

This model has no FIFO ports.

Chapter 8

Formal Parameters

Name	Type	Description
Fundamental		
user_version	Enumeration	Specify required User Architecture version
	2.2	User Architecture Version 2.2
	2.3	Deprecated and equivalent to 20191213
	20190305	Deprecated and equivalent to 20191213
	20191213	User Architecture Version 20191213
priv_version	Enumeration	Specify required Privileged Architecture version
	1.10	Privileged Architecture Version 1.10
	1.11	Privileged Architecture Version 1.11 - ratified 20190608
	1.12	Privileged Architecture Version 1.12 - ratified 20211203
	1.13	Privileged Architecture Version 1.13 - ratified 20241017
	master	Privileged Architecture Master Branch as of commit 2c07aa2 (this is subject to change)
	20190405	Deprecated and equivalent to 1.11
	20190608	Deprecated and equivalent to 1.11
	20211203	Deprecated and equivalent to 1.12
numHarts	Uns32	Specify the number of hart contexts in a multiprocessor
enable_expanded	Boolean	Specify that 48-bit and 64-bit expanded instructions are supported
endianFixed	Boolean	Specify that data endianness is fixed (mstatus.{MBE,SBE,UBE} fields are read-only)
misa_MXL	Uns32	Override default value of misa.MXL
misa_Extensions	Uns32	Override default value of misa.Extensions
add_Extensions	String	Add extensions specified by letters to misa.Extensions (for example, specify "VD" to add V and D features)
sub_Extensions	String	Remove extensions specified by letters from misa.Extensions (for example, specify "VD" to remove V and D features)
misa_Extensions_mask	Uns32	Override mask of writable bits in misa.Extensions
add_Extensions_mask	String	Add extensions specified by letters to mask of writable bits in misa.Extensions (for example, specify "VD" to add V and D features)
sub_Extensions_mask	String	Remove extensions specified by letters from mask of writable bits in misa.Extensions (for example, specify "VD" to remove V and D features)

add_implicit_Extensions	String	Add extensions specified by letters to implicitly-present extensions not visible in misa.Extensions
sub_implicit_Extensions	String	Remove extensions specified by letters from implicitly-present extensions not visible in misa.Extensions
Compressed_Extension		
compress_version	Enumeration	Specify required Compressed Architecture version
	legacy	Compressed Architecture absent or legacy version
	0.70.1	Compressed Architecture Version 0.70.1
	0.70.5	Compressed Architecture Version 0.70.5
	1.0.0-RC5.7	Compressed Architecture Version 1.0.0-RC5.7
	1.0	Compressed Architecture Version 1.0
Interrupts_Exceptions		
rnmi_version	Enumeration	Specify required RNMI Architecture version
	none	RNMI not implemented
	0.2.1	RNMI version 0.2.1
	0.4_nmie1	RNMI version 0.4, except mnstatus.nmie=1 at reset
	0.4	RNMI version 0.4
mtvec_is_ro	Boolean	Specify whether mtvec CSR is read-only
tvec_mode_rsvd	Enumeration	Specify behavior of writes of reserved values to xTVEC.mode field (ignored when CLIC_version <20191208)
	TVEC_M_KEEP	Attempt to set mode to 2 keeps previous value
	TVEC_M_CLIC_VECTOR	Attempt to set mode to 2 will be mapped to 3 (Vectored CLIC mode)
tvec_align	Uns32	Specify additional hardware-enforced alignment on writes to mtvec/stvec/utvec that set a non-direct mode (i.e. bit 0 of the write is 1)
ecode_mask	Uns64	Specify hardware-enforced mask of writable bits in xcause.ExceptionCode
ecode_nmi	Uns64	Specify xcause.ExceptionCode for NMI
nmi_absent	Boolean	Specify whether NMI input is absent
nmi_is_latched	Boolean	Specify whether NMI input is latched on rising edge (if False, it is level-sensitive)
nmi_high_priority	Boolean	Specify whether NMI input is higher priority than Debug and Trigger Module events
nmi_update_mstatus	Boolean	Specify whether an NMI exception updates the mstatus CSR mie and mpie fields
nmi_update_tcontrol	Boolean	Specify whether an NMI exception updates the tcontrol CSR mte and mpte fields
nmi_zero_mtval	Boolean	Specify whether an NMI exception writes 0 to the mtval CSR
mtval_is_ro	Boolean	Specify whether mtval CSR is read-only
tval_zero	Boolean	Specify whether mtval/stval/vstval/utval are hard wired to zero
tval_mask	Uns64	Specify write mask for mtval/stval/vstval/utval. Ignored if tval_zero is True, 0 maps to -1ULL
tval_zero_ecodes_mask	Uns64	Specify mask of exception codes that always set mtval to 0. Not used for interrupts, ignored if tval_zero is True
tval_zero_ebreak	Boolean	Specify whether mtval/stval/vstval/utval are set to zero by an ebreak
tval_ii_code	Boolean	Specify whether mtval/stval contain faulting instruction bits on illegal instruction exception
reset_address	Uns64	Override reset vector address
nmi_address	Uns64	Override NMI vector address

CLINT_address	Uns64	Specify base address of internal CLINT model (or 0 for no CLINT)
local_int_num	Uns32	Specify number of local interrupts (excludes standard set 0 to 15 if basic interrupt mode is present)
unimp_int_mask	Uns64	Specify mask of unimplemented interrupts (e.g. 1<<9 indicates Supervisor external interrupt unimplemented)
force_mideleg	Uns64	Specify mask of interrupts always delegated to lower-priority execution level from Machine execution level
no_ideleg	Uns64	Specify mask of interrupts that cannot be delegated to lower-priority execution levels
no_edeleg	Uns64	Specify mask of exceptions that cannot be delegated to lower-priority execution levels
external_int_id	Boolean	Whether to add nets allowing External Interrupt ID codes to be forced
Fast_Interrupt		
CLIC_version	Enumeration	Specify required CLIC version
	20180831	CLIC Version 20180831 (commit 6d369ab)
	0.9-draft-20191208	CLIC Version 0.9-draft-20191208 (commit f9ab734)
	0.9-draft-20220315	CLIC Version 0.9-draft-20220315 (commit 45a2e91)
	0.9-draft-20221108	CLIC Version 0.9-draft-20221108 (commit fdcd068)
	0.9-draft-20230801	CLIC Version 0.9-draft-20230801 (commit 97a2d57)
	0.10-draft-20250317	CLIC Version 0.10-draft-20250317 (commit 62092fc)
	master	CLIC Master Branch as of commit 62092fc (this is subject to change)
CLICLEVELS	Uns32	Specify number of interrupt levels implemented by CLIC, or 0 if CLIC absent
WorldGuard		
WG_version	Enumeration	Specify required WorldGuard version
	none	WorldGuard not implemented
	0.4	WorldGuard Version 0.4
Debug_Extension		
debug_mode	Enumeration	Specify how Debug mode is implemented (value 'none' implies Sdext is not implemented)
	none	Debug mode not implemented
	vector	Debug mode implemented by execution at vector
	interrupt	Debug mode implemented by interrupt
	halt	Debug mode implemented by halt
	inject	Debug mode implemented using injected instructions
Simulation_Artifact		
leaf_hart_prefix	String	Specify string name prefix for harts in a cluster
mtime_counter	String	Specify name of shared mtime counter
mtime_Hz	Double	Specify mtime counter frequency
mtime_bits	Uns32	Specify mtime counter bit width
use_hw_reg_names	Boolean	Specify whether to use hardware register names x0-x31 and f0-f31 instead of ABI register names
no_pseudo_inst	Boolean	Specify whether pseudo-instructions should not be reported in trace and disassembly
show_c_prefix	Boolean	Specify whether compressed instruction prefix should be reported in trace and disassembly
verbose	Boolean	Specify verbose output messages
traceVolatile	Boolean	Specify whether volatile registers (e.g. minstret) should be shown in change trace
wfi_restart	Boolean	Specify whether to automatically restart from WFI state when model is simulated

enable_CSR_bus	Boolean	Add artifact CSR bus port, allowing CSR registers to be externally implemented
CSR_remap	String	Comma-separated list of CSR number mappings, each of the form <csrName>=<number>
Memory		
unaligned_low_pri	Boolean	Specify whether address misaligned exceptions are lower priority than page or access fault exceptions
unaligned	Boolean	Specify whether the processor supports unaligned memory accesses
Instruction_CSR_Behavior		
wfi_is_nop	Boolean	Specify whether WFI should be treated as a NOP (if not, halt while waiting for interrupts)
wfi_resume_not_trap	Boolean	Specify whether pending wakeup events should cause WFI to be treated as a NOP instead of taking a trap
TW_time_limit	Uns32	Specify nominal cycle timeout for instructions controlled by mstatus.TW
PAUSE_time_limit	Uns32	Specify nominal cycle timeout for PAUSE instruction
counteren_mask	Uns32	Specify hardware-enforced mask of writable bits in mcounteren/scounteren registers
noinhibit_mask	Uns32	Specify hardware-enforced mask of always-zero bits in mcountinhibit register
cycle_undefined	Boolean	Specify that the cycle CSR is undefined
mcycle_undefined	Boolean	Specify that the mcycle CSR is undefined
time_undefined	Boolean	Specify that the time CSR is undefined
instret_undefined	Boolean	Specify that the instret CSR is undefined
minstret_undefined	Boolean	Specify that the minstret CSR is undefined
hpmcounter_undefined	Boolean	Specify that the hpmcounter CSRs are undefined
mhpmcounter_undefined	Boolean	Specify that the mhpmcounter CSRs are undefined
CSR_Masks		
mtvec_mask	Uns64	Specify hardware-enforced mask of writable bits in mtvec register
mip_mask	Uns64	Specify hardware-enforced mask of writable bits in mip register
mtvec_sext	Boolean	Specify whether mtvec is sign-extended from most-significant bit
csrindCustom	Boolean	Force MSB of *iselect CSRs to be writable to support custom extensions using Smcsrind
Trigger		
trigger_num	Uns32	Specify the number of implemented hardware triggers (0 implies Sdtrig is not implemented)
mask_tselect	Boolean	Whether values written to tselect are masked according to the trigger_num setting
tinfo	Uns32	Override tinfo register (for all triggers)
PMP Configuration		
PMP_grain	Uns32	Specify PMP region granularity, G (0 =>4 bytes, 1 =>8 bytes, etc)
PMP_registers	Uns32	Specify the number of supported PMP regions
PMP_csrs	Uns32	Specify the number of PMP address CSRs (are RAZ/WI if corresponding region is not implemented)
PMP_max_page	Uns32	Specify the maximum size of PMP region to map if non-zero (may improve performance; constrained to a power of two)
PMP_decompose	Boolean	Whether unaligned PMP accesses are decomposed into separate aligned accesses

PMP_undefined	Boolean	Whether accesses to unimplemented PMP registers are undefined (if True) or write ignored and zero (if False)
PMP_R0W1	Enumeration	Specify behavior of PMPiCFG RWX field on illegal write with R=0 and W=1
	RWX_00X	set R=0 and W=0, modify X
	RWX_00P	set R=0 and W=0, preserve X
	RWX_11X	set R=1 and W=1, modify X
	RWX_PPX	preserve previous R and W, modify X
	RWX_PPP	preserve previous RWX
	RWX_000	set RWX=000
PMP_A_RSVD	Enumeration	Specify behavior of writes of reserved values to PMPiCFG.A
	PMP_A_NONE	No reserved values for PMPiCFG.A
	PMP_NA4_OFF	Attempts to write NA4 mode to PMPiCFG.A will be mapped to OFF
PMP_maskparams	Boolean	Enable parameters to change the read-only masks for PMP CSRs
PMP_initialparams	Boolean	Enable parameters to change the reset values for PMP CSRs
Sspmp	Boolean	Specify that Sspmp is implemented
mpmpdeleg_min	Uns32	Specify minimum value of mpmpdeleg (if Sspmp enabled)
mpmpdeleg_max	Uns32	Specify maximum value of mpmpdeleg (if Sspmp enabled)
Other_Extensions		
Smstateen	Boolean	Specify that Smstateen is implemented (ignored when priv_version <1.12)
Smcsrind	Boolean	Specify that Smcsrind is implemented (plus Sscsrind if S extension is implemented)
Sscofpmf	Boolean	Specify that Sscofpmf is implemented
Smdbltrp	Boolean	Specify that Smdbltrp is implemented
Smcntrpmf	Boolean	Specify that Smcntrpmf is implemented
Smnpm	Enumeration	Specify Smnpm implemented modes
	none	pointer masking not implemented
	48	PM=XLEN-48 is implemented
	57	PM=XLEN-57 is implemented
	48_57	PM=XLEN-48 and PM=XLEN-57 are implemented
Smmpm	Enumeration	Specify Smmpm implemented modes
	none	pointer masking not implemented
	48	PM=XLEN-48 is implemented
	57	PM=XLEN-57 is implemented
	48_57	PM=XLEN-48 and PM=XLEN-57 are implemented
Zihintntl	Boolean	Specify that Zihintntl is implemented (instruction decode only, implemented as NOP)
Zihintpause	Boolean	Specify that Zihintpause is implemented
Zicond	Boolean	Specify that Zicond is implemented
Zicsr	Boolean	Specify that Zicsr is implemented
Zifencei	Boolean	Specify that Zifencei is implemented
Zicbom	Boolean	Specify that Zicbom is implemented
Zicbop	Boolean	Specify that Zicbop is implemented
Zicboz	Boolean	Specify that Zicboz is implemented
Zimop	Boolean	Specify that Zimop is implemented
Zicfilp	Boolean	Specify that Zicfilp is implemented
Zilsd	Boolean	Specify that Zilsd is implemented
Zibi	Boolean	Specify that Zibi is implemented

ZxlsdPair	Boolean	Specify that Zilsd/Zclsd (if implemented) use two 4-byte memory operations
Zawrs	Boolean	Specify that Zawrs is implemented
Bit_Manipulation_Extension		
misa_B_Zba_Zbb_Zbs	Boolean	Specify that misa.B is present if Zba, Zbb and Zbs are all implemented (from bit manipulation version 1.0.0 only; ignored for previous versions)
Zba	Boolean	Specify that Zba is implemented
Zbb	Boolean	Specify that Zbb is implemented
Zbc	Boolean	Specify that Zbc is implemented
Zbe	Boolean	Specify that Zbe is implemented (ignored if version 1.0.0)
Zbf	Boolean	Specify that Zbf is implemented (ignored if version 1.0.0)
Zbm	Boolean	Specify that Zbm is implemented (ignored if version 1.0.0)
Zbp	Boolean	Specify that Zbp is implemented (ignored if version 1.0.0)
Zbr	Boolean	Specify that Zbr is implemented (ignored if version 1.0.0)
Zbs	Boolean	Specify that Zbs is implemented
Zbt	Boolean	Specify that Zbt is implemented (ignored if version 1.0.0)
CSR_Defaults		
mvendorid	Uns64	Override mvendorid register
marchid	Uns64	Override marchid register
mimpid	Uns64	Override mimpid register
mhartid	Uns64	Override mhartid register (or first mhartid of an incrementing sequence if this is an SMP variant)
mtvec	Uns64	Override mtvec register
AIA_Interrupts		
Smaia	Boolean	Specify that Smaia CSRs are present

Table 8.1: Parameters that can be set in: Hart

8.1 Parameters with enumerated types

8.1.1 Parameter user_version

Set to this value	Description
2.2	User Architecture Version 2.2
2.3	Deprecated and equivalent to 20191213
20190305	Deprecated and equivalent to 20191213
20191213	User Architecture Version 20191213

Table 8.2: Values for Parameter user_version

8.1.2 Parameter priv_version

Set to this value	Description
1.10	Privileged Architecture Version 1.10
1.11	Privileged Architecture Version 1.11 - ratified 20190608
1.12	Privileged Architecture Version 1.12 - ratified 20211203

1.13	Privileged Architecture Version 1.13 - ratified 20241017
master	Privileged Architecture Master Branch as of commit 2c07aa2 (this is subject to change)
20190405	Deprecated and equivalent to 1.11
20190608	Deprecated and equivalent to 1.11
20211203	Deprecated and equivalent to 1.12

Table 8.3: Values for Parameter priv_version

8.1.3 Parameter compress_version

Set to this value	Description
legacy	Compressed Architecture absent or legacy version
0.70.1	Compressed Architecture Version 0.70.1
0.70.5	Compressed Architecture Version 0.70.5
1.0.0-RC5.7	Compressed Architecture Version 1.0.0-RC5.7
1.0	Compressed Architecture Version 1.0

Table 8.4: Values for Parameter compress_version

8.1.4 Parameter rnmi_version

Set to this value	Description
none	RNMI not implemented
0.2.1	RNMI version 0.2.1
0.4_nmie1	RNMI version 0.4, except mnstatus.nmie=1 at reset
0.4	RNMI version 0.4

Table 8.5: Values for Parameter rnmi_version

8.1.5 Parameter CLIC_version

Set to this value	Description
20180831	CLIC Version 20180831 (commit 6d369ab)
0.9-draft-20191208	CLIC Version 0.9-draft-20191208 (commit f9ab734)
0.9-draft-20220315	CLIC Version 0.9-draft-20220315 (commit 45a2e91)
0.9-draft-20221108	CLIC Version 0.9-draft-20221108 (commit fdcd068)
0.9-draft-20230801	CLIC Version 0.9-draft-20230801 (commit 97a2d57)
0.10-draft-20250317	CLIC Version 0.10-draft-20250317 (commit 62092fc)
master	CLIC Master Branch as of commit 62092fc (this is subject to change)

Table 8.6: Values for Parameter CLIC_version

8.1.6 Parameter WG_version

Set to this value	Description
none	WorldGuard not implemented
0.4	WorldGuard Version 0.4

Table 8.7: Values for Parameter WG_version

8.1.7 Parameter debug_mode

Set to this value	Description
none	Debug mode not implemented
vector	Debug mode implemented by execution at vector
interrupt	Debug mode implemented by interrupt
halt	Debug mode implemented by halt
inject	Debug mode implemented using injected instructions

Table 8.8: Values for Parameter debug_mode

8.1.8 Parameter tvec_mode_rsvd

Set to this value	Description
TVEC_M_KEEP	Attempt to set mode to 2 keeps previous value
TVEC_M_CLIC_VECTOR	Attempt to set mode to 2 will be mapped to 3 (Vectored CLIC mode)

Table 8.9: Values for Parameter tvec_mode_rsvd

8.1.9 Parameter PMP_R0W1

Set to this value	Description
RWX_00X	set R=0 and W=0, modify X
RWX_00P	set R=0 and W=0, preserve X
RWX_11X	set R=1 and W=1, modify X
RWX_PPX	preserve previous R and W, modify X
RWX_PPP	preserve previous RWX
RWX_000	set RWX=000

Table 8.10: Values for Parameter PMP_R0W1

8.1.10 Parameter PMP_A_RSVD

Set to this value	Description
PMP_A_NONE	No reserved values for PMPiCFG.A
PMP_NA4_OFF	Attempts to write NA4 mode to PMPiCFG.A will be mapped to OFF

Table 8.11: Values for Parameter PMP_A_RSVD

8.1.11 Parameter Smnpm

Set to this value	Description
none	pointer masking not implemented
48	PM=XLEN-48 is implemented
57	PM=XLEN-57 is implemented
48_57	PM=XLEN-48 and PM=XLEN-57 are implemented

Table 8.12: Values for Parameter Smnpm

8.1.12 Parameter Smmpm

Set to this value	Description
none	pointer masking not implemented
48	PM=XLEN-48 is implemented
57	PM=XLEN-57 is implemented
48_57	PM=XLEN-48 and PM=XLEN-57 are implemented

Table 8.13: Values for Parameter Smmpm

8.2 Parameter values and limits

These are the formal parameter limits and actual parameter values

Name	Min	Max	Default	Actual
Fundamental				
variant				RVI20U32mandatory
user_version			20191213	20191213
priv_version			1.11	1.11
numHarts	0	0x400	0	0
endian			none	none
enable_expanded			f	f
endianFixed			f	f
misa_MXL	1	2	1	1
misa_Extensions	0	0x3ffffff	0x100100	0x100100
add_Extensions				
sub_Extensions				
misa_Extensions_mask	0	0x3ffffff	0x100	0x100
add_Extensions_mask				
sub_Extensions_mask				
add_implicit_Extensions				
sub_implicit_Extensions				
Compressed_Extension				
compress_version			1.0	1.0
Interrupts_Exceptions				
rnmi_version			none	none
mtvec_is_ro			f	f
tvec_mode_rsvd			TVEC_M_KEEP	TVEC_M_KEEP
tvec_align	0	0x10000	0	0
ecode_mask	0	0xffffffffffffffff	0x7fffffff	0x7fffffff
ecode_nmi	0	0xffffffffffffffff	0	0
nmi_absent			f	f
nmi_is_latched			f	f
nmi_high_priority			f	f
nmi_update_mstatus			f	f
nmi_update_tcontrol			f	f
nmi_zero_mtval			f	f

mtval_is_ro			f	f
tval_zero			f	f
tval_mask	0	0xffffffffffffffff	0	0
tval_zero_ecodes_mask	0	0xffffffffffffffff	0	0
tval_zero_ebreak			f	f
tval_ii_code			f	f
reset_address	0	0xffffffffffffffff	0	0
nmi_address	0	0xffffffffffffffff	0	0
CLINT_address	0	0xffffffffffffffff	0	0
local_int_num	0	16	0	0
unimp_int_mask	0	0xffffffffffffffff	0	0
force_mideleg	0	0xffffffffffffffff	0	0
no_ideleg	0	0xffffffffffffffff	0	0
no_edeleg	0	0xffffffffffffffff	0	0
external_int_id			f	f
Fast_Interrupt				
CLIC_version			0.9-draft-20230801	0.9-draft-20230801
CLICLEVELS	0	0x100	0	0
WorldGuard				
WG_version			none	none
Debug_Extension				
debug_mode			none	none
Simulation_Artifact				
leaf_hart_prefix			hart	hart
mtime_counter			mtime_counter	mtime_counter
mtime_Hz	0.000000e+00	1.000000e+09	1.000000e+06	1.000000e+06
mtime_bits	0	64	64	64
use_hw_reg_names			f	f
no_pseudo_inst			f	f
show_c_prefix			f	f
verbose			f	f
traceVolatile			f	f
wfi_restart			f	f
enable_CSR_bus			f	f
CSR_remap				
Memory				
unaligned_low_pri			f	f
unaligned			f	f
Instruction_CSR_Behavior				
wfi_is_nop			f	f
wfi_resume_not_trap			f	f
TW_time_limit	0	0xffffffff	0	0
PAUSE_time_limit	0	0xffffffff	0	0
counteren_mask	0	0xffffffff	0	0
noinhibit_mask	0	0xffffffff	0	0

cycle_undefined			t	t
mcycle_undefined			f	f
time_undefined			t	t
instret_undefined			t	t
minstret_undefined			f	f
hpmcounter_undefined			t	t
mhpmcounter_undefined			f	f
CSR_Masks				
mtvec_mask	0	0xffffffffffffffff	0	0
mip_mask	0	0xffffffffffffffff	0x337	0x337
mtvec_sext			f	f
csrindCustom			f	f
Trigger				
trigger_num	0	255	0	0
mask_tselect			f	f
tinfo	0	0x100ffff	0	0
PMP Configuration				
PMP_grain	0	29	0	0
PMP_registers	0	16	0	0
PMP_csrs	0	64	0	0
PMP_max_page	0	0xffffffff	0	0
PMP_decompose			f	f
PMP_undefined			f	f
PMP_R0W1			RWX_00X	RWX_00X
PMP_A_RSVD			PMP_A_NONE	PMP_A_NONE
PMP_maskparams			f	f
PMP_initialparams			f	f
Sspmp			f	f
mpmpdeleg_min	0	127	0	0
mpmpdeleg_max	0	127	0	0
Other_Extensions				
Smstateen			f	f
Smcsrind			f	f
Sscofpmf			f	f
Smdbltrp			f	f
Smcntrpmf			f	f
Smnpm			none	none
Smmpm			none	none
Zihintntl			f	f
Zihintpause			f	f
Zicond			f	f
Zicsr			f	f
Zifencei			f	f
Zicbom			f	f
Zicbop			f	f

Zicboz			f	f
Zimop			f	f
Zicfilp			f	f
Zilsd			f	f
Zibi			f	f
ZxlsdPair			f	f
Zawrs			f	f
Bit_Manipulation_Extension				
misa_B_Zba_Zbb_Zbs			f	f
Zba			t	t
Zbb			t	t
Zbc			t	t
Zbe			t	t
Zbf			t	t
Zbm			t	t
Zbp			t	t
Zbr			t	t
Zbs			t	t
Zbt			t	t
CSR_Defaults				
mvendorid	0	0xffffffffffffffff	0	0
marchid	0	0xffffffffffffffff	0	0
mimpid	0	0xffffffffffffffff	0	0
mhartid	0	0xffffffffffffffff	0	0
mtvec	0	0xffffffffffffffff	0	0
AIA_Interrupts				
Smaia			f	f

Table 8.14: Parameter values and limits

Chapter 9

Execution Modes

Mode	Code	Description
User	0	User mode
Machine	3	Machine mode

Table 9.1: Modes implemented in: Hart

Chapter 10

Exceptions

Exception	Code	Description
InstructionAddressMisaligned	0	Fetch from unaligned address
InstructionAccessFault	1	No access permission for fetch
IllegalInstruction	2	Undecoded, unimplemented or disabled instruction
Breakpoint	3	EBREAK instruction executed
LoadAddressMisaligned	4	Load from unaligned address
LoadAccessFault	5	No access permission for load
StoreAMOAddressMisaligned	6	Store/atomic memory operation at unaligned address
StoreAMOAccessFault	7	No access permission for store/atomic memory operation
EnvironmentCallFromUMode	8	ECALL instruction executed in User mode
EnvironmentCallFromMMode	11	ECALL instruction executed in Machine mode
MSWInterrupt	67	Machine software interrupt
MTimerInterrupt	71	Machine timer interrupt
MExternalInterrupt	75	Machine external interrupt
GenericNMI	4294967295	Generic NMI

Table 10.1: Exceptions implemented in: Hart

Chapter 11

Hierarchy of the model

A CPU core may be configured to instance many processors of a Symmetrical Multi Processor (SMP). A CPU core may also have sub elements within a processor, for example hardware threading blocks.
OVP processor models can be written to include SMP blocks and to have many levels of hierarchy. Some OVP CPU models may have a fixed hierarchy, and some may be configured by settings in a configuration register. Please see the register definitions of this model.
This model documentation shows the settings and hierarchy of the default settings for this model variant.

11.1 Level 1: Hart

This level in the model hierarchy has 6 commands.
This level in the model hierarchy has 2 register groups:

Group name	Registers
Core	33
Integration_support	3

Table 11.1: Register groups

This level in the model hierarchy has no children.

Chapter 12

Model Commands

A Processor model can implement one or more **Model Commands** available to be invoked from the simulator command line, from the OP API or from the Imperas Multiprocessor Debugger.

12.1 Level 1: Hart

12.1.1 debugflags

show or modify the processor debug flags

Argument	Type	Description
-get	Boolean	print current processor flags value
-mask	Boolean	print valid debug flag bits
-set	Int32	new processor flags (only flags 0x00000006 can be modified)

Table 12.1: debugflags command arguments

12.1.2 getCSRIndex

Return index for a named CSR (or -1 if no matching CSR)

Argument	Type	Description
-name	String	CSR name

Table 12.2: getCSRIndex command arguments

12.1.3 isync

specify instruction address range for synchronous execution

Argument	Type	Description
-addresshi	Uns64	end address of synchronous execution range
-addresslo	Uns64	start address of synchronous execution range

Table 12.3: isync command arguments

12.1.4 itrace

enable or disable instruction tracing

Argument	Type	Description
-access	String	show memory accesses by this instruction. Argument can be any combination of X (execute), A (load or store access) and S (system)
-after	Uns64	apply after this many instructions
-enable	Boolean	enable instruction tracing
-full	Boolean	turn on all trace features
-instructioncount	Boolean	include the instruction number in each trace
-memory	String	(Alias for access). show memory accesses by this instruction. Argument can be any combination of X (execute), A (load or store access) and S (system)
-mode	Boolean	show processor mode changes
-off	Boolean	disable instruction tracing
-on	Boolean	enable instruction tracing
-processorname	Boolean	Include processor name in all trace lines
-registerchange	Boolean	show registers changed by this instruction
-registers	Boolean	show registers after each trace

Table 12.4: itrace command arguments

12.1.5 listCSRs

12.1.5.1 Argument description

List all CSRs in index order

12.1.6 setPMA

Set PMA region permissions and legal access sizes

Argument	Type	Description
-attributes	String	region attributes (string containing r, w, x, a, A, P, 1, 2, 4 or 8)
-hi	Uns64	high address
-lo	Uns64	low address

Table 12.5: setPMA command arguments

Chapter 13

Registers

13.1 Level 1: Hart

13.1.1 Core

Registers at level:1, type:Hart group:Core

Name	Bits	Initial-Hex	RW	Description
zero	32	0	r-	
ra	32	0	rw	
sp	32	0	rw	stack pointer
gp	32	0	rw	
tp	32	0	rw	
t0	32	0	rw	
t1	32	0	rw	
t2	32	0	rw	
s0	32	0	rw	
s1	32	0	rw	
a0	32	0	rw	
a1	32	0	rw	
a2	32	0	rw	
a3	32	0	rw	
a4	32	0	rw	
a5	32	0	rw	
a6	32	0	rw	
a7	32	0	rw	
s2	32	0	rw	
s3	32	0	rw	
s4	32	0	rw	
s5	32	0	rw	
s6	32	0	rw	
s7	32	0	rw	
s8	32	0	rw	
s9	32	0	rw	
s10	32	0	rw	
s11	32	0	rw	
t3	32	0	rw	
t4	32	0	rw	
t5	32	0	rw	
t6	32	0	rw	
pc	32	0	rw	program counter

Table 13.1: Registers at level 1, type:Hart group:Core

13.1.2 Integration_support

Registers at level:1, type:Hart group:Integration_support

Name	Bits	Initial-Hex	RW	Description
commercial	8	0	r-	Commercial feature in use
ASYNCPE	8	0	r-	Asynchronous Event Pending & Enabled
HaltReason	32	0	r-	Bit field indicating halt reason

Table 13.2: Registers at level 1, type:Hart group:Integration_support